

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 2 – Industry Problem Solving Task

Course Code:	CSA06	Course Title:	Design and Analysis of Algorithms
CO Assessed:	CO4: Articulation (BL4, BL5)	Assessment:	Assessment Tool 2 – Industry Problem Solving Task
Weightage:	20% of CIA	Total Marks:	100
CO4 Statement:	Solve optimization problems using dynamic programming and greedy strategies.		
Student Name:	JASMITHA R	Reg. No.:	192524295
Section / Batch:	AI&DS	Date:	22/8/26

Instructions to Students

- Answer the following question. Total Marks: 100.
- Carefully analyze the given questions.
- Explain in detail with sample code if needed.
- Clearly identify all key issues.
- Provide a thorough analysis with validated results and performance evaluation.

Question Type	Focus Area	Questions
Industry Problem Solving Task	Focus on Solving optimization problems using dynamic programming and greedy strategies	Q1

SECTION A – Industry Problem Solving Task

Code of Conduct

I JASMITHA R(192524295)(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: JASMITHA R

Q15. Network Packet Routing Optimization (Shortest Path Algorithms) An internet service provider must route data packets across a large network efficiently to minimize latency. Analyze how shortest path algorithms like Dijkstra or Bellman-Ford can be applied. Identify constraints such as network congestion and dynamic edge weights. Design a solution for real-time routing optimization. Discuss scalability for large networks. Evaluate time complexity and memory usage. Interpret performance in high-speed communication systems.

1. Introduction

Every time a person streams a video, sends an email, or loads a webpage, the data is broken into small units called packets. These packets do not travel to their destination directly through a single wire; instead, they hop through a large mesh of interconnected routers and switches owned by different Internet Service Providers (ISPs) across cities, countries, and continents. Deciding which sequence of routers a packet should pass through is called routing, and the quality of this decision directly determines how fast a webpage loads, how smooth a video call is, and how reliable an online transaction feels. Because the underlying physical network can be modeled as a mathematical graph, where routers are nodes and physical or logical links are edges, the problem of choosing the best path naturally becomes a shortest path problem, and this is exactly where classical graph algorithms such as Dijkstra's Algorithm and the Bellman-Ford Algorithm become extremely important in real-world engineering.

This answer explains, in depth, how these two algorithms work, why they are suited to network routing, what real constraints (like congestion and changing link quality) complicate their use, how a real-time routing optimization system can be designed around them, how such a system scales to very large networks, what their time and memory costs are, how they behave in high-speed communication systems, and finally how the same ideas show up in everyday real-life systems outside computer networks.

2. Modeling the Network as a Graph

Before applying any shortest path algorithm, the physical network must first be represented in a form the algorithm can understand. This representation is called a graph, written as $G = (V, E)$, where V is the set of vertices (routers or network devices) and E is the set of edges (physical links or logical connections between routers).

- Vertices (V): Represent routers, switches, or autonomous systems (in the case of the wider internet).
- Edges (E): Represent physical cables, wireless links, or virtual tunnels connecting two devices.
- Edge Weight (w): A numeric cost assigned to each edge, representing how 'expensive' it is to send a packet across that link. This can be based on latency, distance, bandwidth, monetary cost, or a combination of factors.

Once the network is expressed this way, finding the best route from a source router S to a destination router D becomes the same mathematical problem as finding the shortest path between two nodes in a graph, which is precisely what Dijkstra's and Bellman-Ford's algorithms are designed to solve.

3. How Dijkstra's Algorithm Works

Dijkstra's Algorithm finds the shortest path from a single source node to every other node in a graph, provided that all edge weights are non-negative. It works using a greedy strategy: at every step, it picks the unvisited node with the smallest known distance from the source, finalizes that distance, and then relaxes (updates) the distances of its neighboring nodes if a shorter path is found through it.

Step-by-step working:

- Initialize the distance to the source node as 0, and the distance to every other node as infinity.
- Place all nodes in a priority queue (min-heap), ordered by their current known distance.
- Repeatedly extract the node with the smallest distance from the priority queue.
- For each neighbor of that node, calculate the distance through the current node. If this is smaller than the neighbor's currently known distance, update it (this is called 'relaxation').
- Repeat until the priority queue is empty or the destination node has been finalized.

Simplified pseudocode:

```
function Dijkstra(Graph, source):
    dist[source] = 0
    for each vertex v != source: dist[v] = infinity
    priorityQueue.add(all vertices, key = dist)
    while priorityQueue is not empty:
```

```

    u = extract vertex with minimum dist
    for each neighbor v of u:
        if dist[u] + weight(u, v) < dist[v]:
            dist[v] = dist[u] + weight(u, v)
            previous[v] = u
    return dist, previous

```

In routing, this algorithm is the mathematical engine behind link-state protocols such as OSPF (Open Shortest Path First) and IS-IS. Every router first floods information about its directly connected links to all other routers so that every router ends up with an identical, complete map (link-state database) of the network. Each router then independently runs Dijkstra's Algorithm on this map, using itself as the source, to build a shortest-path tree to every other router. From this tree, it extracts only the 'next hop' for every destination and stores it in its forwarding table.

4. How Bellman-Ford's Algorithm Works

Bellman-Ford's Algorithm also finds shortest paths from a single source to all other nodes, but unlike Dijkstra, it can correctly handle graphs that contain negative edge weights (as long as there is no negative-weight cycle). Instead of greedily picking the closest node like Dijkstra, it takes a brute-force but systematic dynamic programming approach: it relaxes every single edge in the graph, repeatedly, for $(V - 1)$ rounds, where V is the number of vertices.

Step-by-step working:

- Initialize the distance to the source as 0, and to every other node as infinity.
- Repeat the following $(V - 1)$ times: for every edge (u, v) with weight w , if $\text{dist}[u] + w < \text{dist}[v]$, update $\text{dist}[v] = \text{dist}[u] + w$.
- After $(V - 1)$ rounds, perform one more pass over all edges: if any distance can still be improved, this indicates a negative-weight cycle exists in the graph.

Simplified pseudocode:

```

function BellmanFord(Graph, source):
    dist[source] = 0
    for each vertex v != source: dist[v] = infinity
    repeat (V - 1) times:
        for each edge (u, v, weight) in Graph:
            if dist[u] + weight < dist[v]:
                dist[v] = dist[u] + weight
    for each edge (u, v, weight) in Graph:

```

```

        if dist[u] + weight < dist[v]:
            report 'negative cycle detected'
    return dist

```

In routing, Bellman-Ford's underlying idea (the distance-vector approach) is used by protocols like RIP (Routing Information Protocol). Instead of every router knowing the entire network, each router only knows its direct neighbors and the distance it has calculated to reach every destination. Routers periodically exchange these distance vectors with their neighbors, and each router updates its own table if a neighbor advertises a shorter path. Over several rounds, this distributed relaxation process converges to the correct shortest paths, which is essentially Bellman-Ford's Algorithm running in a decentralized, message-passing form across the whole network rather than inside a single machine.

5. Comparing Dijkstra and Bellman-Ford

Feature	Dijkstra	Bellman-Ford
Edge weights	Non-negative only	Can handle negative weights
Approach	Greedy, priority queue	Dynamic programming, relaxation
Time Complexity	$O((V+E) \log V)$ with min-heap	$O(V \times E)$
Used in	OSPF, IS-IS (link-state)	RIP (distance-vector)
Knowledge required	Full network topology (global view)	Only immediate neighbors (local view)
Convergence	Fast, but needs a centralized/global map	Slower, but naturally distributed
Weakness	Cannot handle negative weights	Slow; prone to 'count-to-infinity' loops on failures

In short, Dijkstra is preferred when a router (or a central controller) can build a full, accurate map of the network and needs fast, one-time computation. Bellman-Ford, or its distance-vector philosophy, is preferred when routers only have local information and must cooperate to arrive at a shared understanding of the network gradually.

6. Constraints in Real Networks

Textbook versions of Dijkstra and Bellman-Ford assume a static graph with fixed edge weights, but real networks are far messier. The following constraints must be understood and handled before these algorithms can be trusted in production routing systems.

- **Network Congestion:** A shortest path measured purely by hop count or physical distance may pass through a heavily congested router or link. Even though it is 'shortest' on paper, the actual delay experienced by packets can be far higher than a slightly longer but less congested alternative path. Congestion is also dynamic: a link that is free at one moment can be saturated seconds later due to sudden traffic bursts.
- **Dynamic Edge Weights:** In real networks, edge weights are not fixed numbers; they must be recalculated continuously based on real-time measurements such as latency, jitter, packet loss rate, and available bandwidth. This means the graph itself is constantly changing, and any shortest path computed a moment ago may no longer be optimal.
- **Scalability of Large Topologies:** The global internet has hundreds of thousands of routable network blocks and an enormous number of interconnected routers. Recomputing a full shortest-path tree over such a massive graph every time a single link changes is computationally very expensive and would never finish fast enough to be useful.
- **Path Asymmetry:** The best path from router A to router B is not always the same as the best path from B back to A, because ISPs apply different routing policies, business agreements, and load patterns in each direction.
- **Fault Tolerance and Loops:** Links and routers fail unexpectedly due to hardware faults, fiber cuts, or power outages. When this happens, algorithms must reconverge to a new set of correct shortest paths quickly. Distance-vector approaches like RIP are particularly vulnerable here to a problem called 'count-to-infinity', where routers keep increasing their advertised distance to a now-unreachable destination in small increments because they are unknowingly relying on each other's outdated information, creating a routing loop.
- **Limited Router Memory and Processing Power:** Real routers, especially at the network edge, have limited CPU and memory. This restricts how large a topology they can store, and how frequently they can afford to recompute shortest paths without affecting live traffic forwarding.

7. Designing a Real-Time Routing Optimization Solution

Given the constraints above, a naive, one-time application of Dijkstra or Bellman-Ford is not sufficient for a real ISP-scale network. A practical, real-time routing optimization system layers several additional techniques on top of the base shortest path algorithm.

7.1 Dynamic, Multi-Factor Edge Weighting

Instead of using a single static number like hop count, each edge weight is computed as a weighted combination of multiple real-time metrics, so that the 'shortest' path also reflects the 'healthiest' path:

```
weight(edge) = alpha * latency + beta * (1 /  
available_bandwidth)  
              + gamma * congestion_factor + delta *  
packet_loss_rate
```

Here, alpha, beta, gamma, and delta are tunable coefficients that let network engineers prioritize, for example, low latency for voice/video traffic or high bandwidth for bulk file transfers. These weights are refreshed periodically using data collected from routers via monitoring protocols such as SNMP or telemetry streaming.

7.2 Incremental / Dynamic Shortest Path Updates

Rather than recomputing the entire shortest-path tree from scratch every time one edge weight changes, real systems use incremental (dynamic) shortest path algorithms. These algorithms detect which part of the previously computed shortest-path tree is affected by the change and only recompute that local region, leaving the rest of the tree untouched. This drastically reduces computation time on large graphs where only a small part of the network has actually changed.

7.3 Hierarchical Routing

Large networks are divided into smaller regions (called 'areas' in OSPF, or 'Autonomous Systems' at the internet scale). Dijkstra's Algorithm is run at the local level inside each small region, while a separate, coarser algorithm handles routing between regions. This means no single computation ever has to process the entire global network graph at once, which keeps each individual computation fast even though the overall network is huge.

7.4 Load-Aware Rerouting with Damping

The system periodically re-evaluates routes using updated congestion data, but includes hysteresis (a form of damping) so that routes do not change too frequently in response to small, temporary fluctuations. Without this, routers could 'flap' between two nearly equal paths constantly, wasting processing power and destabilizing the network.

7.5 Equal-Cost Multi-Path (ECMP) Load Balancing

When shortest path computation reveals multiple paths of identical or near-identical cost, instead of committing all traffic to just one of them, the router splits traffic across all of them. This spreads load more evenly, reduces the chance of any single path becoming a bottleneck, and improves overall network utilization.

7.6 Centralized Optimization via Software-Defined Networking (SDN)

In modern data centers and large-scale ISP backbones, a centralized SDN controller collects real-time topology and traffic information from every router (often via protocols like OpenFlow or BGP-LS), runs an optimized shortest-path computation centrally with a complete, up-to-date global view (similar to Dijkstra's requirement), and then pushes updated forwarding rules to routers. This combines the accuracy of a globally-aware algorithm with the responsiveness needed for real-time optimization, something traditional fully-distributed protocols struggle to achieve as quickly.

8. Scalability for Large Networks

As a network grows from a handful of routers to millions of connected devices, naive shortest-path computation becomes a serious bottleneck. Several strategies make the algorithms scale to real-world, internet-sized networks.

- **Hierarchical/Area-Based Division:** By splitting the network into OSPF areas or BGP autonomous systems, each shortest-path computation only has to consider a small subset of the overall network, rather than the entire internet.
- **Sparse Graph Representation:** Real networks are sparse, meaning each router connects to only a small number of other routers relative to the total network size. Using adjacency lists instead of adjacency matrices to represent the graph saves enormous amounts of memory and speeds up traversal.
- **Distributed Computation:** Distance-vector-style protocols spread the computational burden across every router in the network instead of relying on one central machine to compute everything, which naturally scales as more routers are added.
- **Caching and Incremental Updates:** Previously computed shortest paths are cached and only selectively updated when a relevant part of the network changes, instead of being recomputed completely on every single update.
- **Route Aggregation / Summarization:** Multiple smaller destination address blocks are combined into a single summarized route advertisement wherever possible, reducing the number of entries routers must store and process.

- **Policy-Based and Heuristic Routing at Internet Scale:** At the scale of the global internet, pure shortest-path optimization is often replaced with policy-based routing (as in BGP), where business agreements between ISPs, and heuristics rather than pure mathematical distance, determine path selection, because pure shortest-path computation across the entire internet is not practically feasible.

9. Time Complexity and Memory Usage

Algorithm	Time Complexity	Space Complexity
Dijkstra (array-based)	$O(V^2)$	$O(V + E)$
Dijkstra (binary heap)	$O((V+E) \log V)$	$O(V + E)$
Dijkstra (Fibonacci heap)	$O(E + V \log V)$	$O(V + E)$
Bellman-Ford	$O(V \times E)$	$O(V)$

- **Array-based Dijkstra:** Simple to implement but inefficient on large, sparse graphs because it scans all nodes at every step to find the minimum, giving $O(V^2)$ time.
- **Binary heap Dijkstra:** Using a min-heap priority queue significantly speeds up minimum extraction, giving $O((V+E) \log V)$, which is efficient for the sparse graphs typical of real router networks.
- **Fibonacci heap Dijkstra:** Provides the theoretically fastest known time complexity for Dijkstra, $O(E + V \log V)$, though it is more complex to implement and carries higher constant-factor overhead, so it is used mainly in performance-critical, very large-scale systems.
- **Bellman-Ford:** Simpler in structure but slower overall at $O(V \times E)$, since it must relax every edge across $(V - 1)$ full passes; however, it uses less memory per node since each router only stores information about its direct neighbors.

In practice, memory usage is just as important as time complexity for router hardware. Physical routers use specialized, expensive, and limited memory called TCAM (Ternary Content-Addressable Memory) to store forwarding tables for extremely fast lookups. Because TCAM capacity is limited, techniques like route summarization and hierarchical addressing (CIDR) are essential to keep the number of stored routes manageable, even though this is a hardware constraint that pure algorithmic time-complexity analysis does not capture.

10. Performance in High-Speed Communication Systems

In backbone networks built on fiber-optic cables and high-speed data-center interconnects, data travels at speeds close to the speed of light, and even a delay of a few microseconds can matter for latency-sensitive applications like high-frequency trading, live video conferencing, or online gaming. This creates specific performance expectations for routing algorithms in such environments.

- **Pre-computed Forwarding, Not Per-Packet Computation:** Shortest path algorithms are not run for every individual packet, as that would be far too slow. Instead, they run periodically or on topology change events to build forwarding tables in advance, and actual packet forwarding uses simple, extremely fast hardware-based table lookups.
- **Convergence Speed After Failure:** A key performance metric is how quickly the network 'reconverges' to correct, loop-free shortest paths after a link or router failure. Link-state protocols based on Dijkstra generally converge faster than distance-vector protocols based on Bellman-Ford, which is one reason ISP backbones favor OSPF and IS-IS over RIP.
- **Trade-off Between Optimality and Responsiveness:** Continuously recalculating routes using the very latest congestion data improves accuracy but adds computational load and can cause instability if updates happen too frequently; well-designed systems balance freshness of data against the stability and responsiveness of the network as a whole.
- **Hardware Acceleration:** High-speed routers use dedicated hardware (ASICs) for the actual packet-forwarding decision, while the shortest-path computation itself typically runs on the router's control-plane CPU separately, so that route computation never slows down the actual forwarding of packets already in transit.

11. Comparison with Real Life

The mathematics behind packet routing is not unique to computer networks; the same underlying shortest-path thinking appears throughout everyday systems that humans interact with regularly.

- **GPS Navigation (Google Maps, Waze):** Just like OSPF, GPS navigation apps use Dijkstra or A* variants on a road-network graph where edge weights (road segments) are continuously updated using real-time traffic congestion data, directly mirroring congestion-aware edge weighting in packet routing. When a road is blocked, the app recomputes a new shortest path in real time, exactly as a router recomputes a new forwarding path after a link failure.

- **Postal and Courier Delivery Networks:** Distance-vector-style routing resembles how local post offices or courier depots only know their immediate neighboring hubs and forward a parcel to whichever neighbor gets it closer to its destination, similar to how RIP-based routers only know their direct neighbors rather than the entire network.
- **Airline Route Planning (Hub-and-Spoke):** Airlines use hierarchical, hub-and-spoke routing, similar to how the internet is divided into autonomous systems and OSPF areas. Flights are routed through major hub airports to keep the overall route network manageable, rather than operating direct point-to-point flights between every pair of cities.
- **Traffic Signal and Road Rerouting During Accidents:** When an accident closes a road, navigation apps recompute alternate routes dynamically, just as routers recompute paths when a network link fails. Both systems face the identical core engineering challenge: recomputing a new optimal path fast enough to remain genuinely useful, without excessive delay or computational overhead.
- **Electric Power Grid Distribution:** Power grid operators similarly model substations and transmission lines as a graph and use shortest-path and flow optimization techniques to route electricity efficiently while balancing load across different lines, closely paralleling ECMP-style load balancing in packet networks.
- **Supply Chain and Logistics Networks:** Large logistics companies model warehouses and distribution centers as nodes and transport routes as edges, using shortest-path and flow algorithms to decide how goods should move through the network at minimum cost and time, which is conceptually identical to minimizing latency in packet routing.

12. Conclusion

Dijkstra's Algorithm and Bellman-Ford's Algorithm form the mathematical backbone of how modern computer networks decide where to send data. Dijkstra powers fast, globally-aware link-state protocols like OSPF, while Bellman-Ford's relaxation-based philosophy underlies simpler, distributed distance-vector protocols like RIP. Neither algorithm, in its pure textbook form, is sufficient on its own for a real, large-scale, constantly changing network; practical routing systems must add dynamic, multi-factor edge weighting, incremental recomputation, hierarchical division of the network, load balancing, and often centralized SDN-based optimization to handle congestion, failures, and scale in real time. The time and memory cost of these algorithms directly shapes what hardware and architecture

choices are feasible, and their behavior under high-speed conditions determines how responsive and stable a network feels to its users. Ultimately, the same shortest-path thinking that keeps the internet running efficiently is mirrored in many everyday systems people rely on, from GPS navigation to airline scheduling to postal delivery, showing that the core idea of finding the best path through a network of connected points is one of the most broadly useful concepts in computer science and beyond.

RUBRICS FOR CALCULATION

Industry Problem Solving Task

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%				
Application of Engineering Concepts	25%				
Solution Design	25%				
Use of Tools	15%				
Analysis	10%				
Professionalism	5%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name: _____	Name: _____	Name: _____
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

